The IvPHelm as a MOOS Application

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/chapters/chap_helm_as_moos`

Contents

1 The IvP Helm as a MOOS Application

In this section the helm is discussed in terms of its identity as a MOOS application - its MOOS configuration parameters, its `Iterate()` loop, its output to the console, and its output in terms of publications to other applications running in the MOOS community. The *helm state* and *all-stop status* are also introduced since they are the highest level descriptions regarding helm activity.

1.1 Overview

The IvP Helm is implemented as the MOOS module called `pHelmIvP`. On the surface it is similar to any other MOOS application - it runs as a single process that connects to a running `MOOSDB` process interfacing solely by a publish-subscribe interface, as depicted in Figure 1. It is configured from a behavior file, or `.bhv` file, in addition to the MOOS file used to configure other MOOS applications. The helm primarily publishes a steady stream of information that drives the platform, typically regarding the desired heading, speed or depth. It may also publish information conveying aspects of the autonomy state that may be useful for monitoring, debugging or triggering other algorithms either within the helm or in other MOOS processes. The helm can be configured to generate decisions over virtually any user-defined decision space.

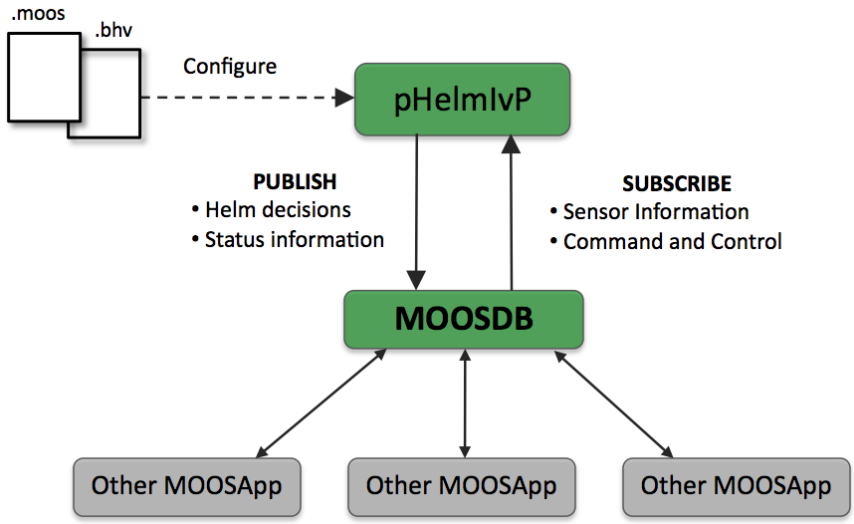

Figure 1: **The pHelmIvP MOOS application**: The IvP Helm is implemented as the MOOS application `pHelmIvP`. The helm is configured with two files - the mission file and behavior file. Once launched it connects to the MOOSDB along with other MOOS applications performing other functions. Information flowing into the helm include both sensor information and command and control inputs. The helm produces commands for maneuvering the vehicle along with other status information produced by active behaviors.

The helm subscribes for sensor information or any other information it needs to make decisions. This information includes navigation information regarding the platform's current position and trajectory, information regarding the position or state of other vehicles, or environmental information. The information it subscribes for is prescribed by the behaviors themselves, configured in the `.bhv` file. In addition to sensor information, the helm also receives some level of command and control information. For example, in some marine vehicle configurations, one of the "Other MOOSApp" modules in the figure is a driver for an acoustic modem over which command and control information may be relayed.

The helm has a couple informative high-level state descriptions, the *helm state* and the *all-stop status*, that may be compared to the operation of an automobile. Launching the pHelmIvP MOOS process is analagous to turning on the car's engine. Putting the helm in the `DRIVE` mode is like shifting the car from "Park" to "Drive". And the all-stop status refers whether or not the car is breaking to a stop. The analogy is summarized below.

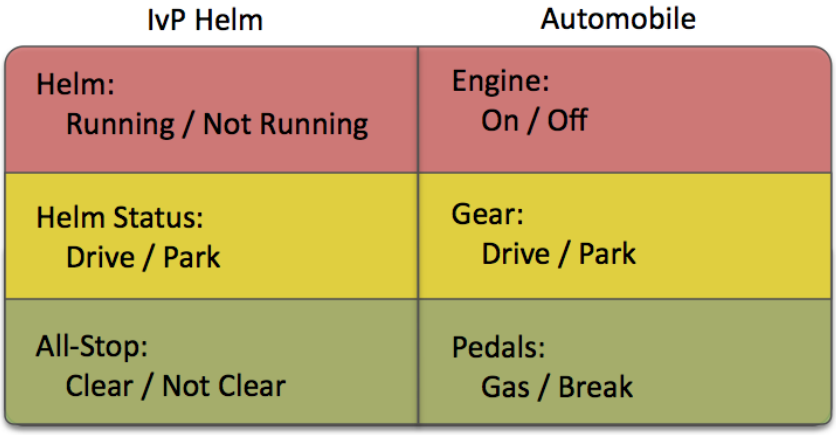

Figure 2: **The Helm/Automobile Analogy**: The helm and its high-level state descriptions, the *helm state* and *all-stop status*, have analogies with the operation of an automobile.

1.2 The Helm State

The highest level interface with the helm is reflected in the *helm state*. The helm state may have one of two values, *park* or *drive* (except when using a *standby helm* described in Section 1.7) In the *drive* mode, the helm is likely in the process of executing a mission. In the *park* mode, the helm is waiting, likely because it is being asked to wait, but also because the helm may have noticed something wrong (generated an all-stop) and subsequently put itself into *park* on its own, awaiting a the chance to return to the *drive* state. In this section we discuss (a) how the helm state is changed, (b) what it is going on in the helm when it is parked, and (c) how the helm state is initialized at start-up. At any point in time after the helm is launched, the helm will post the MOOS variable `IVPHELM_STATE` with either the value `"DRIVE"`, `"PARK"` or `"MALCONFIG"`. This is posted on each iteration and registering for this mail is the manner recommended by which other MOOS applications monitor the helm's heart beat.

1.2.1 Helm State Transitions

The helm state may be transitioned by writing to the MOOS variable `MOOS_MANUAL_OVERIDE`. As Figure 3 depicts, a value of `false`, which is case insensitive, transitions the helm state into `DRIVE`. A value of `true` puts it into the `PARK`. When the helm transitions from `DRIVE` to `PARK` it makes *one* more publication to the helm decision variables, each with a value zero. This is referred to as the production of an *all-stop posting*, discussed in more detail in a later section.

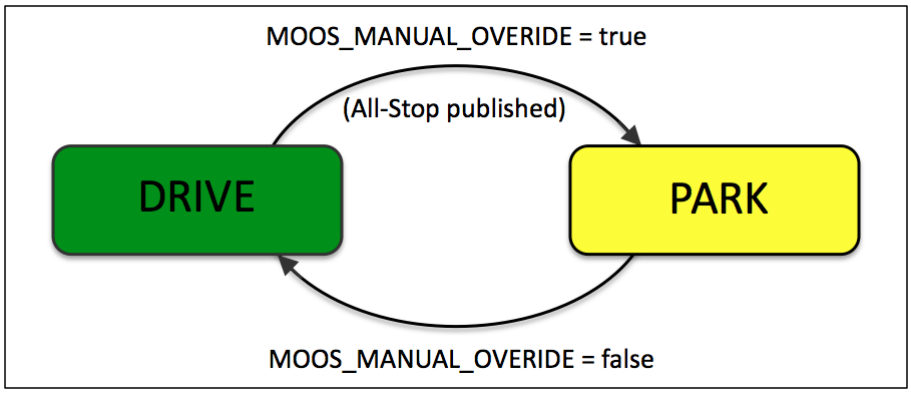

Figure 3: **The Helm State of the IvP Helm**: The helm state has a value of either PARK or DRIVE, depending on both how the helm is initialized and the mail received by the helm after start-up on the variable MOOS_MANUAL_OVERIDE. The helm may also park itself if an all-stop event has been detected.

The variable MOOS_MANUAL_OVERIDE contains the mis-spelling of "override". However, it is a variable that has some legacy presence in other MOOS applications such as iRemote. To avoid a situation where there is an attempt to override the helm, but the request is ignored because of a (proper) spelling, the helm will also respect transition requests on the properly spelled variable MOOS_MANUAL_OVERRIDE. This has the drawback however that these two variables could conceivably have different values in the MOOSDB. This is not a problem but could be confusing for someone trying to infer the helm state by opening a scope on the MOOSDB, on either the wrong variable or the two disagreeing variables. In this case the helm state would be aligned with the variable with the most recent publication time stamp. In any event, the best way to monitor the helm state is to scope on the MOOS variable IVPHELM_STATE, published by the helm itself, or use the uHelmScope tool.

The helm may also automatically transition itself from the DRIVE to the PARK state (but never the other way around), by posting an all-stop event. All-stop events and the helm are discussed separately in Section 1.3. All-stop events are generated by the helm upon finding that one or more possible error conditions have been detected during the normal execution of the helm iteration. If the helm parks due to an all-stop, it may be returned to the DRIVE state by another MOOS client posting MOOS_MANUAL_OVERIDE=false, but this is no guarantee that the helm wouldn't just park again immediately if the same condition persists that caused the all-stop event.

1.2.2 What Is and Isn't Happening when the Helm is Parked

When the helm state is PARK, the MOOS application loop carries on. The OnNewMail() continues to be called and new mail is read and dealt with exactly as it would if the helm state were DRIVE. The Iterate() loop, however, is truncated to virtually a no-op, with the only action being the output of a heartbeat character to the console if the helm is configured to do so (Section 1.5). No behavior code is called whatsoever. The helm iteration counter, a key index in the uHelmScope output, is also suspended despite the fact that technically the Iterate() loop continues to be called.

1.2.3 Initializing the Helm State at Process Launch Time

The helm, by default, is configured to be initiallly in PARK upon start-up. By setting the parameter start_in_drive=true in the mission file configuration block, the helm will indeed be in the DRIVE upon start-up. This feature was found to have practical use in UUV operations to allow for rebooting of the autonomy computer to automatically launch the helm, in DRIVE and ready to accept field commands. This feature should be used with caution, and it may be phased out in a later software release.

1.3 Helm All-Stop Events and All-Stop Status

An *all-stop event* is something that brings the vehicle to a full-stop, with typically zero-speed, zero-depth commanded. The *all-stop status* is simply a string describing *why* the vehicle is at an all-stop. Sometimes an all-stop indicates a problem, e.g., missing critical sensor information. Sometimes a vehicle may just stop as part of the mission, e.g., coming to the surface for a GPS fix. The all-stop status message can be used to discern the two types of situations. In the automobile analogy, an all-stop is equivalent to hitting the car's breaks with the intent to stop completely. An all-stop event will result in the following:

- Zero-values will be posted for all decision variables, DESIRED_SPEED=0, DESIRED_DEPTH=0, etc.
- The helm will possibly transition into PARK. By default the helm is configured to remain in the DRIVE upon an all-stop event, but if configured instead with park_on_allstop = true, the helm will indeed park upon an all-stop.
- The reason for the all-stop will be posted to MOOS variable IVPHELM_ALLSTOP. The value of this variable will be "clear" if there are no all-stop events that have occurred since the helm has entered the DRIVE state.

The reasons for all-stop may be:

- No behaviors are active. The helm has absolutely no opinion about any of its decision variables. In this case, the following would be posted: IVPHELM_ALLSTOP ="NothingToDo".
- Some behaviors are active, but decisions are missing on one or more mandatory decision variables. In this case, the following would be posted: IVPHELM_ALLSTOP ="MissingDecVars".
- When the vehicle is parked due to manual override, the following would be posted: IVPHELM_ALLSTOP ="ManualOverride".
- One of the behaviors has determined an all-stop is warranted for some reason. For example, a waypoint behavior that cannot determine own-platform's current position would declare an all-stop. In this case, the following would be posted: IVPHELM_ALLSTOP ="BehaviorError".

To gain further insight on an all-stop caused by a behavior error, the nature of the error is expressed in a separate posting to the MOOS BHV_ERROR variable. It's possible that more than one behavior error occurred in the helm iteration where the all-stop event occurred, in which case there would be multiple postings to the BHV_ERROR variable. When the vehicle is in DRIVE and operating free of an all-stop, the all-stop status is reflected by IVPHELM_ALLSTOP ="clear".

1.4 Parameters for the pHelmIvP MOOS Configuration Block

The following configuration parameters are defined for the helm. The parameter names are case insensitive.

Listing 1.1: Configuration Parameters for pHelmIvP.

allow_park:	If false, the helm cannot be put into PARK. This parameter is not mandatory. The default is true. Section 1.4.1.
behaviors:	The name and location of the behavior configuration file. This parameter is not mandatory, but typically it is used. Technically the helm can be launched from the command line and provided the behavior file on the command line. Section 1.4.2.
ivp_behavior_dir:	A directory to look for dynamically loaded behaviors. This parameter is not mandatory, since the directory information may also be handled using a shell environment variable. Section 1.4.3.
community:	Global MOOS parameter. Determines ownship name. This parameter is mandatory, but it is provided outside the helm configuration block and used by other applications. Section 1.4.4.
park_on_allstop:	If true helm will park on all-stop. This parameter is not mandatory and the default is false. Secttion 1.4.1.
domain:	The decision space for the IvP Solver. This parameter is mandatory. Section 1.4.6.
hold_on_app:	A list of MOOS apps to wait for before the helm publishes postings from behaviors' onHelmStart() function calls. Available after Release 17.7.x. Section 1.4.10.
ok_skew:	The tolerance on the age, in seconds, of incoming mail before rejected as being too old. This parameter is not mandatory. Section 1.4.7.
other_override_var:	The parameter names an additional MOOS variable acting as MOOS_MANUAL_OVERIDE. This parameter is not mandatory. Section 1.4.8.
start_in_drive:	Determines whether or not the helm is in override mode at start-up. This parameter is not mandatory. The default is false. Secton 1.4.9.
helm_prefix:	Add a prefix to all the DESIRED_* helm output. For example helm_prefix=FOO_ would result in FOO_DESIRED_SPEED and so on. Introduced after Release 19.8.x
verbose:	Determines verbosity of terminal output - quiet, terse, or verbose. This parameter is not mandatory. The default is verbose. Section 1.4.11.

1.4.1 The allow_park Parameter

Optional. By setting this parameter to false, the helm cannot be manually overriden (parked) once it has been put into DRIVE. This can be dangerous and should be carefully considered, and thus the default is true. This option was implemented based on experiences with launching UUV autonomy missions and preventing an inadvertent park due to a remote login to the vehicle. There was a

tendency for some users to use iRemote upon remote login to interact with MOOS, and iRemote posts MOOS_MANUAL_OVERIDE =true upon launch and connection to the MOOSDB.

1.4.2 The behaviors Parameter

Optional (sort of). The parameter names the behavior file, i.e., *.bhv file, on the local file system from which the helm behaviors are read. More than one file may be specified on separate lines, and the helm will read in all files almost as if they were one single file. This is technically an optional parameter because a behvior file could be provide on the command line. A behavior file must be specified via one means or the other. If a behavior file is specified *both* on the command line and in the pHelmIvP configuration block with this parameter, they will both be used to configure the helm behaviors.

1.4.3 The behavior_dir Parameter

Optional. The parameter names a directory in the local files system where the helm is to look for dynamically loadable behaviors (as opposed to default set built in statically to the helm). Authors augmenting the helm with their own behaviors will need to specify the location of those behaviors with this parameter. More than one line may be provided, each specifying a different directory location.

1.4.4 The community Parameter

This parameter is defined at the "global" level outside of any MOOS process configuration block. The helm reads this parameter and uses its value as the name associated with "ownship". It is a mandatory parameter.

1.4.5 The park_on_allstop Parameter

Optional. By setting this parameter to true, the helm will park when an all-stop event occurs. The default setting is false.

1.4.6 The domain Parameter

Mandatory. This parameter prescribes the decision space of the helm. It consists of one line per decision variable. Each line contains a colon-separated list of four fields. Field one is the domain variable name, field two is the lower bound value, field three is the higher bound value, and field four is the number of points in the domain. For example domain = speed:0:3:16 indicates a domain variable called "speed", with a lower and upper bound 0 and 3 meters/second respectively. Since there are 16 points, the speed choices are 0, 0.2, 0.4, ..., 2.8, 3.0. The helm requires that a decision be made on all listed variables on each iteration of the control loop. If a variable is used by some behaviors but is not necessarily involved in all decisions, it can be declared as optional. For example domain=speed:0:3:16:optional.

1.4.7 The ok_skew Parameter

Optional. This parameter sets the allowable skew tolerated by the helm for receiving incoming mail messages. If a clock skew is detected greater than this value, the message will be ignored. A check for skews can be disabled by setting `ok_skew=any`. The default value is 60 seconds.

1.4.8 The other_override_var Parameter

Optional. This parameter names a MOOS variable the helm will regard as being synonomous with the two default variables accepted for manual override, `MOOS_MANUAL_OVERRIDE`, and the legacy mispelling of this variable, `MOOS_MANUAL_OVERIDE`.

1.4.9 The start_in_drive Parameter

Optional. This parameter is set to either `true` or `false`. The default is `false` as the helm normally starts in `PARK` and needs to receive MOOS mail on the variable `MOOS_MANUAL_OVERIDE` with the value of this variable set to `false`. When `start_in_drive` is set to `true`, the helm is in the `DRIVE` state upon start-up. The issue of helm state was discussed in more detail in Section 1.2.

1.4.10 The hold_on_app Parameter

Optional. Available after Release `17.7.x`. This parameter names one or more MOOS apps that the helm will monitor in the `DB_CLIENTS` list. Once it has seen each named app at least once, the helm will release all mail generated by behaviors through their `onHelmStart()` function.

This may be useful for behaviors that produce configuration information to other support applications. Examples include the `pBasicContactMgr` and `pTaskManager` application. These apps need to be told by the behaviors the nature of alerts that may be generated, potentially resulting in spawned behaviors. If the behaviors produce multiple postings of the same MOOS variable, and those postings occur before the intended recipient apps have come on line (connected to the MOOSDB), then they may only receive the last piece of mail. With this utility, the helm will wait until all named apps are connected before posting the `onHelmStart()` mail.

The parameter may take a comma-separated list of apps, or multiple lines may be provided. The following two styles are equivalent:

```
hold_on_app = pBasicContactMgr, pTaskManager
```

and

```
hold_on_app = pBasicContactMgr
hold_on_app = pTaskManager
```

1.4.11 The verbose Parameter

Optional. This parameter affects how much information is written to the terminal on each iteration of the helm. The possible values are *verbose*, *terse*, or *quiet*. The *verbose* setting will write a brief helm report to the terminal on each iteration. With the *terse* setting minimal output will be produced, a '*' character when not producing helm commands, and a '$' character when active and healthy. With the *quiet* setting, no output at all will be written to the terminal. The default value

is *terse*. This setting can be changed after the helm is started by changing the value of HELM_VERBOSE in the MOOSDB.

1.4.12 An Example pHelmIvP MOOS Configuration Block

Below is an example configuration block for the helm.

Listing 1.2: An example pHelmIvP *configuration block.*

```
//-------- pHelmIvP configuration block  -------------
ProcessConfig = pHelmIvP
{
  AppTick    = 4   // Defined for all MOOS processes
  CommsTick  = 4   // Defined for all MOOS processes

  domain     = course:0:359:360
  domain     = speed:0:3:16
  domain     = depth:0:500:101

  behaviors  = foobar.bhv
  verbose    = terse
  ok_skew    = ANY

  start_in_drive  = false
  allow_park      = true
  park_on_allstop = false
}
```

The AppTick and CommsTick parameters are defined for all MOOS processes and specify the frequency in which the helm process iterates and communicates with the MOOSDB. The community parameter is not included in the configuration block because it is specified at the global level in the mission file.

1.5 Launching the Helm and Output to the Terminal Window

The helm can be launched either directly from the command line, or from within Antler. On the command line the usage is as follows:

```
Usage: pHelmIvP file.moos [file.bhv]...[file.bhv]
       [--help|-h] [--version|-v]

[file.moos] Filename to get MOOS config parameters.
[file.bhv]  Filename to get IvP Helm config paramters.
[-v]        Output version number and exit.
[-h]        Output this usage information and exit.
```

If no behavior file is specified in the .moos file then a behavior file must be given on the command line. Multiple behavior files may be provided. Order of the arguments do not matter - command line arguments ending in .bhv will be read as behavior files, and those ending with .moos as MOOS files. The specification of behavior files may also be split between references in the .moos file and the command line. The duplicate specification of a single file will simply be ignored. Typical start-up output to the terminal is shown in Listing 3 below.

Listing 1.3: Example start-up output generated by the pHelmIvP process.

```
 0  ************************************************
 1  *                                              *
 2  *       This is MOOS Client                    *
 3  *       c. P Newman 2001                       *
 4  *                                              *
 5  ************************************************
 6
 7  ---------------MOOS CONNECT----------------------
 8    contacting a MOOS server localhost:9000 -  try 00001
 9    Contact Made
10    Handshaking as "pHelmIvP"
11    Handshaking Complete
12    Invoking User OnConnect() callback...ok
13  -----------------------------------------------
14
15  The IvP Helm (pHelmIvP) is starting....
16  Loading behavior dynamic libraries....
17      Loading directory: /Users/mikerb/project-colregs/src/lib_behaviors-colregs
18  Loading behavior dynamic libraries - FINISHED.
19  Number of behavior files: 1
20  Processing Behavior File: bravo.bhv  START
21      Successfully found file: bravo.bhv
22      InitializeBehavior: found static behavior BHV_Loiter
23      InitializeBehavior: found static behavior BHV_Loiter
24      InitializeBehavior: found static behavior BHV_Waypoint
25      InitializeBehavior: found static behavior BHV_Timer
26  Processing Behavior File: bravo.bhv  END
27  pHelmIvP is Running:
28    AppTick   @ 4.0 Hz
29    CommsTick @ 4 Hz
30    Time Warp @ 1.0
31  $$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
```

The output in lines 0-13 are standard output generated by a MOOS process launched and successfully connected to a running MOOSDB. Lines 15-30 are start-up output generated unique to the helm and the particular user usage. Behaviors used by the helm are either static or dynamic. Static behaviors are compiled in to the pHelmIvP executable. Dynamic behaviors are brought in at run time via shared libraries compiled separately. The helm looks for an environment variable IVP_BEHAVIOR_DIRS for a colon-separated list of directories to search for shared libraries. If this variable is not set, or if one or more of the directories are not legitimate directories, an error message will indicate so between what is otherwise line 16 and 18 in Listing 3. This kind of error may not actually be problematic if the behaviors specified in the behavior file can all be otherwise successfully found.

For each specified behavior file, the information shown in lines 20-26 is generated to the terminal. For each behavior configuration in a given .bhv file, a single line is output as in lines 22-25 indicating that the behavior type is recognized and it is configured properly. A single unrecognized behavior or improper configuration will result in (a) an error message indicating the offending line number and file name, (b) the output of the actual offending line, and (c) immediate disconnection of the process from the MOOSDB and exit. (Tip: If the helm is launched with Antler an error during start-up will result in the closing of the pHelmIvP console window which makes it hard to catch useful error output for debugging. In this case, the helm should just be launched outside of Antler in its own terminal window.)

The output on line 31 of Listing 3, a series of dollar sign characters, indicates for each character,

the completion of a single helm iteration - a heartbeat output. This is the output when the verbose parameter is set to the default setting of terse. When set to quiet no output is generated at all. When set to verbose, a short multi-line report is generated for each iteration. An example is shown below in Listing 4:

Listing 1.4: An example helm iteration report generated by an active helm.

```
 1  Iteration: 161  ****************************************
 2  Helm Summary  --------------------------
 3  loiter_a did NOT produce an obj-function
 4  loiter_b produces obj-function - time:0.00 pcs: 9.00000 pwt: 100.00000
 5  waypt_return did NOT produce an obj-function
 6  loiter_timer did NOT produce an obj-function
 7  Number of Objective Functions: 1
 8  DESIRED_SPEED:  2.10
 9  DESIRED_COURSE:  145.00
10  (End) Iteration: 161  ****************************************
```

On each iteration the Helm Summary indicates which behaviors produced objective functions (lines 2-5), and for those that did, it indicates the CPU time needed to generate the function, the number of pieces in the piecewise linear IvP function, and its priority weight. Following this, the decision rendered for current iteration is output with one line per decision variable (lines 7-8). This is a very thin summary of what is going on within the helm and it should be noted that the uHelmScope tool is a much better suited for monitoring helm activity and debugging.

1.6 Publications and Subscriptions for IvP Helm

The IvP Helm, like any MOOS process, can be specified in terms of its interface to the MOOSDB, i.e., what variables it publishes and what variables it subscribes for. It is impossible to provide a complete specification here since the helm is comprised of behaviors, and the means to include any number of third party behaviors. Each behavior is able to post variable-value pairs, published to the MOOSDB by the helm on behalf of the behavior at the end of the iteration. Likewise, each behavior may declare to the helm any number of MOOS variables it would like the helm to register for on its behalf. Barring these variables, published and subscribed for by the helm on behalf of individual behaviors, this section addresses the remaining portion of the helm's publish - subscribe interface.

1.6.1 Variables published by the IvP Helm

Variables published by the IvP Helm are summarized below.

- IVPHELM_SUMMARY: Produced on each iteration of the helm for consumption by the uHelmScope application. It contains information on the current helm iteration regarding the number of IvP functions created, create time, solve time, which behaviors are active, running, idle, and the decision ultimately produced during the iteration. The summary does not include every component in each summary. Components that have not changed in value since the prior summary are dropped from the present summary. This is motivated by the goal to reducing the log file footprint for the helm.
- IVPHELM_STATEVARS: Produced periodically by the helm for consumption by the uHelmScope application. It contains a comma-separated list of MOOS variables involved in preconditions of any behavior, i.e., variables affecting behavior run states.

- IVPHELM_DOMAIN: Produced once by the helm at start-up for consumption by the uHelmScope application. It contains the specification of the IvP Domain in use by the helm.
- IVPHELM_MODESET: Produced once by the helm at start-up for consumption by the uHelmScope application. It contains the specification of the Hierarchical Mode Declarations, if any, in use by the helm.
- IVPHELM_STATE: Written by the helm on each iteration of the pHelmIvP MOOS application, regardless of whether the helm is in the DRIVE state or not. (see Section 1.2). It is either "DRIVE" or "PARK". This is the recommended MOOS variable for regarding as a "heartbeat" indicator of the helm.
- HELM_IPF_COUNT: Produced on each iteration of the helm. It contains the number of IvP functions involved in the solver on the current iteration.
- CREATE_CPU: The CPU time in seconds used in total by all behaviors on the current iteration for constructing IvP functions.
- LOOP_CPU: The CPU time in seconds used by the IvP solver in the current helm iteration.
- BHV_IPF: The helm will publish this variable for each active behavior in the current iteration. It contains a string representation of the IvP function produced by the behavior. It is used for visualization by the uFunctionVis application, and for logging and later playback and analysis.
- PLOGGER_CMD: This variable is published with the below value to ensure that the pLogger application logs the .bhv file along with the other data log files and the .moos file.

```
"COPY_FILE_REQUEST = filename.bhv"
```

- DESIRED_*: Each of the decision variables in the IvPDomain provided in the helm configuration will have a separate posting prefixed by DESIRED_ as in DESIRED_SPEED. One exception is that the variable course will be converted to heading for legacy reasons.
- BHV_WARNING: Although this variable may never be posted, it is the default MOOS variable used when a behavior posts a warning. A warning may be harmless but deserves consideration.
- BHV_ERROR: Although this variable may never be posted, it is the default MOOS variable used when a behavior posts what it considers a fatal error - one that the helm will interpret as a request to generate the equivalent of ALL-STOP.

In addition to the above variables, the helm will post any variable-value pair on behalf of a behavior that makes the request. These include endflags, runflags, idleflag, activeflags and inactiveflags.

1.6.2 Variables Subscribed for by the IvP Helm

Variables subscribed for by the IvP Helm are summarized below:

- MOOS_MANUAL_OVERIDE: When set to true, usually by a third-party application such as iRemote, of from a command-and-control communication, the helm may relinquish control. If the helm was configured with active_start = true, it will not relinquish control (this may be changed).
- HELM_VERBOSE: Affects the console output produced by the helm. Legal values are verbose, terse, or quiet. See Section 1.5.
- HELM_MAP_CLEAR: When received, the helm clears an internal map that is used to surpress repeated duplicate postings. See Section 1.8.

In addition to the above variables, the helm will subscribe for any variable-value pair on behalf of a behavior that makes the request. This includes, but is not limited to, variables involved in the `condition` and `updates` parameters available generally for all behaviors.

1.7 Using a Standby Helm

A *standby helm* refers to the launching of a second helm running alongside another otherwise normally-configured primary helm. This is done to mitigate the risk associated with the possible failure of the primary helm. Both helms are instances of the `pHelmIvP` MOOS application configured with a different mission and/or behaviors. Presumably the standby helm is configured with a simpler, more conservative set of behaviors focused on the safe recovery of a vehicle. Although provisions are generally made during vehicle operations to detect a missing helm heartbeat, in situations such as the operation of a UUV under ice, it is not acceptable to simply have a vehicle halt and come to the surface. An attempt should be made to execute a simpler mission to return the vehicle to a safe location for recovery.

Use of a standby helm is as simple as adding a second configuration block in the `.moos` configuration file. The Kilo example mission demonstrates a mission using the standby helm. Declaring a second helm as a standby helm requires a single additional line of the form

```
STANDBY = N
```

where `N` is the number of seconds a standby helm will tolerate an absent primary helm heartbeat before taking control from away the primary helm. *Once a standby helm take-over is triggered, the take-over is irreversible.*

1.7.1 Two Types of Helm Failure, the Causes, and Detection

What does a helm crash mean, and how might it happen? A crash is result of code that causes the process to quit unexpectedly and without warning. A line as simple as `assert(0);` in the code would be sufficient to replicate a crash, but the cause of a crash could be much more subtle due referencing memory not properly allocated and so on. The other type of helm failure is a helm that *hangs*. This refers to the scenario where the helm enters a piece of code that takes too long or never finishes execution. This could be as trivial as reaching the line `while(1);` somewhere in the code. Either type of failure is serious. Despite the fact that we have never experienced these failures in any field exercise on any vehicle, the sudden disappearance of the helm process should be considered and handled as gracefully as possible.

How is a helm failure detected? The helm produces a heartbeat message on each iteration by posting to the variable `IVPHELM_STATE`. If the helm is configured to iterate four times per second, suspicion of failure could begin anytime after a quarter of a second passes without a posting to this variable. Under typical helm CPU load with common standard behaviors, the quarter second interval should be sufficient to finish the iteration. There are additional factors to consider however. There may be other processes on the machine dominating the CPU, thus challenging the helm to do its work in the expected time. There may be a periodic behavior calculation, such as recalculating a long path of waypoints, that may cause a spike in CPU cycles needed by the behavior. As a rule of thumb, the interval of time with the absence of a heartbeat, should arguably be two seconds or more before

declaring a helm failure. This interval is directly configurable, as the parameter N, in the standby=N configuration line.

1.7.2 Handling a Helm Crash with the Standby Helm

A helm *crash* is the easier of the two failure cases to handle. The process is simply gone and no longer publishes to the MOOSDB. Of course the other MOOS processes, including the standby helm, have no way of knowing simply through their MOOS mailbox whether or not the primary helm is gone or just delayed. In either case the sequence of events is the following:

1. The standby helm detects the absence of heartbeat for more than N seconds.
2. On the very same iteration, the standby helm posts IVPHELM_STATE = DRIVE+ rather than IVPHELM_STATE = STANDBY which it had been posting on every iteration up until now. It also begins posting a desired helm decision on this iteration.

The standby helm has all the same functionality as the primary helm, modulo the behavior and mission configuration. It will be in the DRIVE state only if not manually overridden. If MOOS_MANUAL_OVERRIDE=true when the standby helm takes over, it will be in initially in PARK, not DRIVE. It will post IVPHELM_STATE=PARK+. Note the standby helm will append the '+' character to the helm state string to help other applications and the user discern that the helm state being posted is from a standby helm that has taken control.

The Kilo example mission walks through a simulated helm crash.

1.7.3 Handling a Hung Helm with the Standby Helm

Handling a helm that has *hung* requires a bit more consideration. The tricky part is that quite possibly the hung helm is only *temporarily* hung, and at some point it will become unhung and may operate for a single iteration as if it is still the helm in charge. Two helms, with different missions, both thinking they are in charge! The sequence of events is summarized below:

1. The standby helm detects the absence of heartbeat for more than N seconds.
2. On the very same iteration, the standby helm posts IVPHELM_STATE = DRIVE+ rather than IVPHELM_STATE = STANDBY which it had been posting on every iteration up until now. It also begins posting a desired helm decision on this iteration.
3. Some time later, the original primary helm finishes its iteration and posts a helm decision, e.g., a set of postings to the helm decision variables, DESIRED_HEADING etc.
4. On the next iteration of the original primary helm it notices that another helm has posted a non-standby heartbeat, e.g., a posting to IVPHELM_STATE not equal to "STANDBY".
5. The original primary helm posts an all-stop and IVPHELM_STATE = DISABLED. It is the last time it will post a heartbeat or helm decision.
6. The standby helm continues to be in control posting helm decisions oblivious to the former primary helm's epiphany and temporary influence.

The key issue in this scenario is that the original primary helm does indeed post a helm decision when it becomes un-hung even though the standby helm may have long ago taken over and may be posting a sequence of helm decisions completely at odds with the decision posted by the newly awakened un-hung primary helm.

No assumptions can be made about the MOOS process listening to the sequence of helm decisions. It may be payload interface process, or a native PID controller, or some other process responsible for converting helm decisions to lower level actuator commands. The assumption that *is* made here is that a one-time aberation in the sequence in helm decisions is tolerable. This aberation is not going to result in an actuator breaking due to a sudden change in the command sequence such as high-speed, zero-speed, high-speed.

It's also worth noting that original primary helm, upon awakening and learning it is no longer in control, does indeed post one more helm decision, an all-stop decision (`DESIRED_SPEED = 0`). This is done to ensure that an all-stop decision, that may have been put into effect by the standby helm, is not superceded by the output of the original primary helm's final helm decision made upon wakeup.

1.7.4 Activity of the Standby Helm While Standing By

In the standby state the helm will do nothing in its iterate loop other than post a heartbeat character to the terminal if configured to do so. It will not call on any behaviors to do anything. The helm will however read and process all of its otherwise subscribed for mail. In particular it will monitor for postings to `IVPHELM_STATE`, and will initiate a take-over when it hasn't received such mail for `N` seconds. Note the standby helm also publishes to this variable, `IVPHELM_STATE = STANDBY`, but ignores mail originating from itself.

The helm will also read and process all other mail in its inbox, updating the helm's information buffer. Note that the information buffer also keeps a *history* of postings to a particular variable so that normally a behavior may process multiple postings if multiple postings were made to the `MOOSDB` between helm iterations. This history is cleared by the helm at the end of each helm iteration. When the helm is in the standby state it will also clear the history after each iteration even though the behaviors have not been given the chance to access this history. This is to ensure that the information buffer history doesn't grow without bound while in standby mode. For example, if the standby and primary helm are both configured with a waypoint behavior and the primary helm visits half the points at the point of a helm crash, the standby helm's waypoint behavior would start with the first waypoint.

1.7.5 Activity of the Primary Helm After Take-Over

A primary helm that has been taken over is refered to as a *didsabled* helm. It will post only once `IVPHELM_STATE = DISABLED`. The helm process lives on however doing next to nothing. It does not read its mail, and the iterate loop simply posts a heartbeat character (`'!'`) to the terminal if configured to do so, with the helm configuration parameter `verbose=terse`.

1.8 Automated Filtering of Successive Duplicate Helm Publications

The helm implements a "duplication filter" to drastically reduce the amount of mail posted by the helm on behalf of behaviors. This filter has been noted to reduce the overall log file size seen during

in-water exercises by 60-80%. Reductions at this level noticably facilitate the use of post-mission analysis tools and data archiving. For the most part this filter is operating behind the scenes for the typical helm user. However, knowledge of it is indeed relevant for users wishing to implement their own behaviors, and we discusss it here to explain a bit what is behind the variable HELM_MAP_CLEAR to which the helm subscribes, and listed above in Section 1.6.2.

1.8.1 Motivation for the Duplication Filter

The primary motivation of implementing the duplication filter is to reduce the amount of unnecessary mail posted by the helm on behalf behaviors, and thereby greatly reduce the size of log files and facilitate the post-mission handling of data. By unnecessary we mean successive variable-value pairs that match exactly in both fields. Surely there are cases when a behavior developer may not want this filter, and there are simple ways to bypass the filter for any post. But in most cases, successive duplicate posts are just redundant and unnecessary.

1.8.2 Implementation and Usage of the Duplication Filter

The helm keeps two maps (STL maps in C++), one for string data and one for numerical data:

```
KEY --> StringValue
KEY --> DoubleValue
```

The two maps correspond to the double and string message types in MOOS. The KEY is typically the MOOS variable name. Inside a behavior implementation, the following four functions are available:

```
void postMessage(string varname, string value, string key="");
void postMessage(string varname, double value, string key="");
void postBoolMessage(string varname, bool value, string key="");
void postIntMessage(string varname, double value, string key="");
```

These functions are available in all behavior implementations because they are defined in the IvPBehavior superclass, of which all behaviors are subclasses. Before the helm posts a message to the MOOSDB the filter is applied by a simple check to its map to determine if there is a value match on the given key. If a match is made, the post will not be made to the MOOSDB on the behavior's behalf. The postIntMessage() function is merely a convenience version of the postMessage() function that rounds the variable value to the nearest integer to further reduce posts when combined with the filter. The postBoolMessage() ultimately posts a string value "true" or "false".

The default value of the *key* parameter is the empty string, and in most cases this parameter can be ommitted without disabling the duplication filter. This is because the KEY used by the caller is only part of the key actually used by the duplication filter. The actual key is the concatenation of (a) the behavior name, (b) the variable name, and (c) the key passed by the caller. Thus the default value, the empty string, still results in a decent key being used by the filter. The key is augmented by the behavior name because often there is more than one behavior posting messages on same variable. The optional key parameter is used for two reasons. First, it can be used to further distinguish posts within a behavior on the same variable name. Second, when the key value has the special value "repeatable", then no key is used and the duplication filter is disabled for that variable posting.

Two additional convenience functions are availble:

```
void postRepeatableMessage(string varname, string value);
void postRepeatableMessage(string varname, double value);
```

A posting of `postRepeatableMessage("FOO", 100)` is equivalent to `postMessage("FOO", 100, "repeatable")`.

1.8.3 Clearing the Duplication Filter

Occasionally a user, or another MOOS application in the same community as the helm, may want to "clear" the map used by the helm to implement its duplication filter. This can be done by writing to variable HELM_MAP_CLEAR, with any value. This may be necessary for the following reason. Suppose a GUI application subscribes for the variable VIEW_SEGLIST which contains a list of line segments for rendering. If the viewer application is launched *after* the variable is published, the application will only receive the most recent mail on the variable VIEW_SEGLIST. There may be publications to this variable, made prior to the most recent publication, that are relevant to the GUI application at launch time. Those publications for the variable VIEW_SEGLIST may not be the most recent from the perspective of the MOOSDB, but they may be the most recent from the perspective of a particular behavior in the helm. By clearing the filter, it gives each behavior the chance to once again have all of its variable-value posts made to the MOOSDB. In the pMarineViewer application, a publication to HELM_MAP_CLEAR is made upon start-up. Clearing the filter will only clear the way for the next post for a given variable. It will not result in the publishing to the MOOSDB of the contents of the maps used by the filter.